Analytics Vidhya · Follow publication

NLP + Data Science ✓ Machine Learning + Deep Learning + Statistics +

# The Cold-Start Problem with Stanford Snorkel

6 min read · Feb 8, 2020

Austin Powell Follow

Listen Share ••• More


This is a brief write-up on my practical experience using Stanford Snorkel. It is intended as an introduction to the problem space, motivation for using it and sharing what practical experience I gained.

## Introduction

### What is it?

Very briefly, Stanford Snorkel is a Python library to help you label data for supervised machine learning tasks.

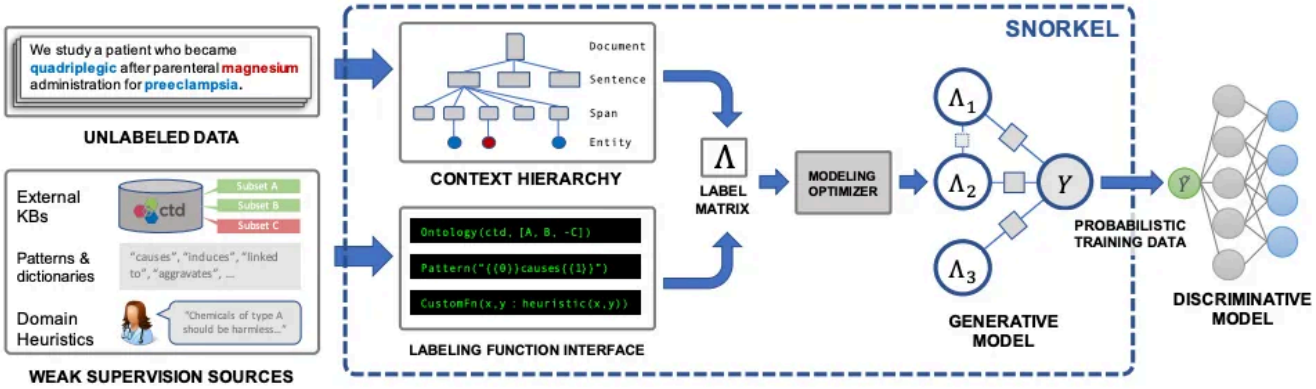



Diagram from the group's published paper: https://arxiv.org/pdf/1711.10160.pdf

### Why use it?

The short answer should be very obvious: you would like to estimate some *y-hat* given some data *X*. But I need *y* in order to train, so what do I don't when I don't have *y?* This is typically referred to as the "cold start problem" and is there is a growing appetite to make use of the massive amounts of data being accumulated. The most straightforward and most practical solution is to label the data.

If it were just me, myself and I like with my actual motivation for starting on this project it's fairly simple. Say I want to create model to decide whether email is spam or not. I'll sit down for 1hr and label a bunch of data. In order to scale this, a lot of companies use Mechanical Turk where you give instructions to a bunch of humans to sit down and label a bunch of data.



### How to use it?

Writing the labeling functions is a bit art and science. This advice comes from both experience and the creators of Snorkel. Below are some of the tips that seemed to work well in my use-case:

- *Write a balance of labeling functions:* If you are writing labeling functions for a binary classifier, you want to balance out the negative examples with the positive ones. Even though you are using weak learners, the goal is to place equal importance on the labels for generative model.
- *Don't be afraid of writing conflicting labels:* Your goal isn't to write amazingly logical functions just insightful ones.
- *Try to write labeling functions with high coverage:* This means that for text rules where you are looking for female gendered words in primarily male gendered text, your converge will be low.
- *Have a golden hold-out set:* The labeling functions are not humans. Only you really know what labels should be. If you aren't able to define the difference between a positive and negative tweet, how can you expect the same of the labeler?

### Applying in real life

Much of the information covered so far can be gleaned by reading on the well-written tutorials on your own. My goal for taking the time investigate this however was the net trade-off for utilizing Snorkel.

Could it get me from zero → lot of labels → something could use better than I could be going through them by hand?



*My use case is what is referred to as Authorship Attribution (attributing text to the correct author). In this case, a message to a clinician belonging to a parent or a teen. The teen is supposed to be the only one who is accessing clinician information but parent's tend to use this feature anyway.*

> *One way of measuring effort might be to say how much time it took to empirically to create the labeling functions only. For me,* ***it took a few hours to get some intuition on how to***

> ***create labeling functions**, but most of that time was getting used to the workflow. I believe it would go quicker next time since I would know how to create and evaluate.*

The steps I took:

- 1) Hand-label ~100 messages for “gold standard”
- 2) Create labeling a few labeling functions and iterate through 3) and 4)
- 3) Validate as I’m writing the functions with a reasonable coverage and conflict in a validation test.
- 4) I stopped when my “golden standard” set reached 90% using the labeling functions
- 5) Use the model you can train with Snorkel to test on final hold-out test set to see what the prod-environment “labeling” might look like.

A few take-aways:

- Often the majority-label rule wound up being more accurate on “golden standard” than the probabilistic Snorkel method.
- I was able to get > 90% after training a model on the Snorkel labeled dataset and testing on test set, but unclear as to whether this was a net benefit to just labeling more by hand.
- Snorkel give some evaluative measures such as *label conflict, polarity,* and *coverage* that are helpful diagnostics to finding labels that give a good signal as you label your data.
- I feel Snorkel would be more beneficial to multi labelers where the intuition on the data or what the true label is (I’m thinking Mechanical Turk here) might be a bit more “fuzzy”.

### Some Code-Labeling Functions

Here are the labeling functions for me that wound up getting me started.

- **Tabular labeling functions**

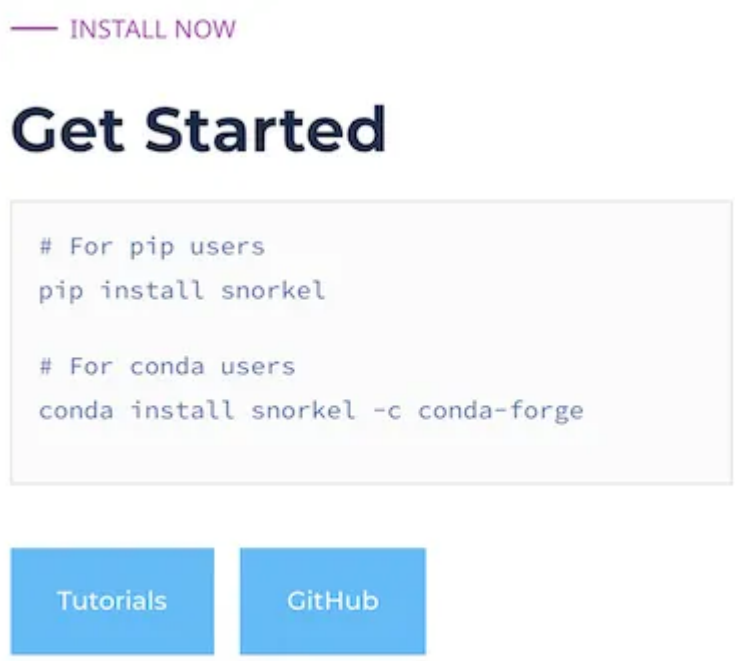
```
@labeling_function()
def child_in_msg(x):
    """Postive examples (unknown otherwise) if child's name is in message.
    """
    return 1 if x.pat_first_in_msg == 1 else -1


@labeling_function()
def child_is_older(x):
    """Positive and negative examples for different cut-offs for child's age.
    """
    if x.age < 15.5:
        return 1
    elif 15.5 <= x.age < 17:
        return -1
    elif 16 <= x.age:
        return 0


@labeling_function()
def email_domain(x):
    """Gmail is a strong signal for not being a parent in text.
    """
    return 0 if "gmail" in x.email_domain else -1


@labeling_function()
def is_proxy(x):
    """Positive example (0 otherwise) if a proxy account.
    """
    return 1 if x.proxy_account_yn == "Y" else 0


@labeling_function()
def email_domain2(x):
    """Oracle is not a email domain easily accessable to teens.
    """
    return 1 if "oracle" not in x.email_domain else -1
```

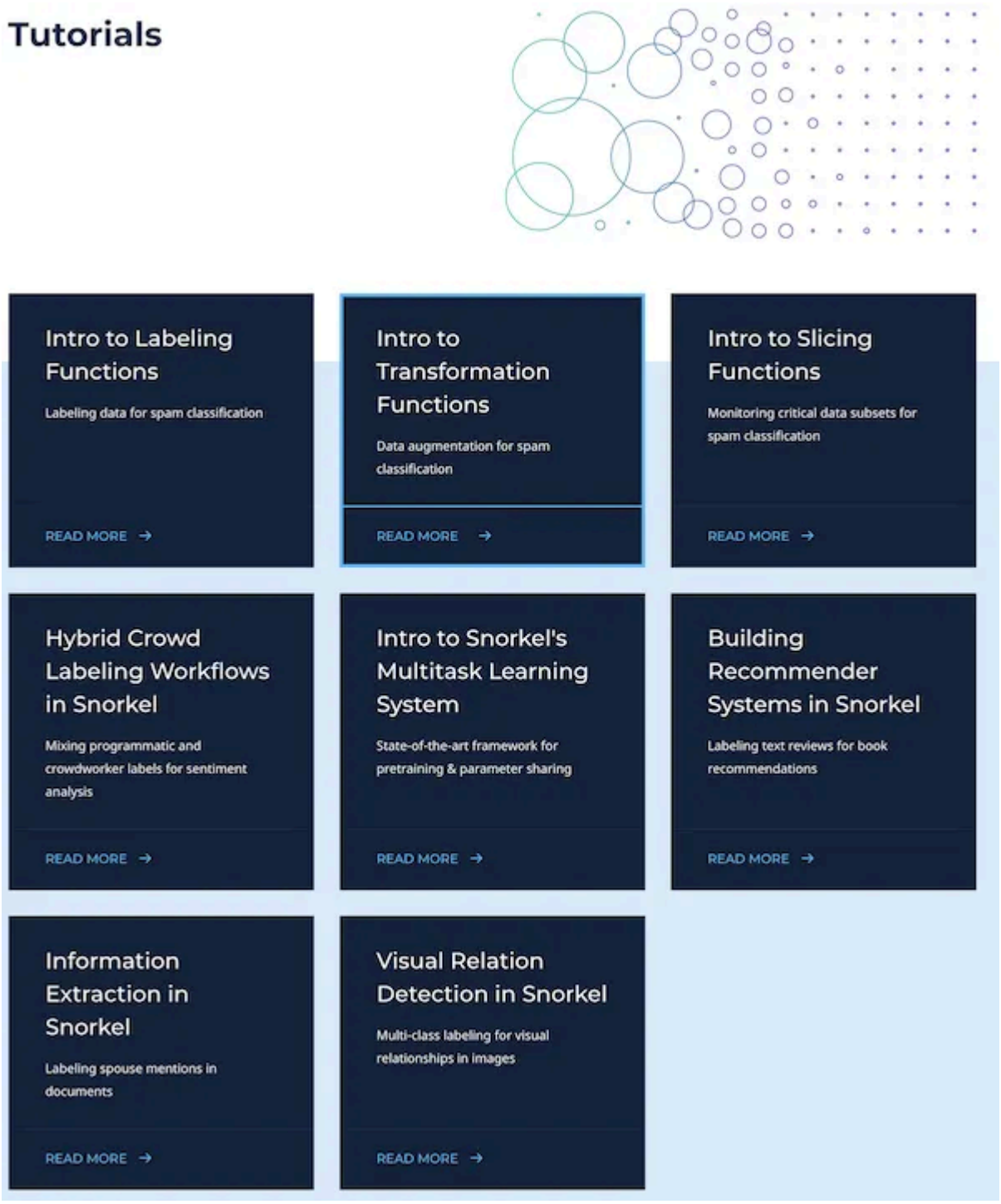
- **Text labeling functions** These were applied to the text corpus. In some ways this is the best use-case for snorkel since you can be pretty free and loose with your hypothesis. I noticed it definitely makes a difference in final result when you make sure to balance positive with negative labeling functions.

```python
from snorkel.labeling import LabelingFunction


def keyword_lookup(x, keywords, label):
    if any(word in x.msg_txt.lower() for word in keywords):
        return label
    return -1


def make_keyword_lf(keywords, label=1):
    return LabelingFunction(
        name=f"keyword_{keywords[0]}",
        f=keyword_lookup,
        resources=dict(keywords=keywords, label=label),
    )


######################################################################
# Positive or unknown keywords
######################################################################
keyword_my_son = make_keyword_lf(keywords=["son"])

keyword_my_daughter = make_keyword_lf(keywords=["my daughter"])

keyword_my_child = make_keyword_lf(keywords=["my child", "your child"])


keyword_son = make_keyword_lf(keywords=["son", "he", "him"])

keyword_referring = make_keyword_lf(keywords=["for us", "our"])

keyword_daughter = make_keyword_lf(keywords=["daughter", "she", "her"])

keyword_child = make_keyword_lf(keywords=["child", "parent", "child's"])

######################################################################
# Negative or unknown keywords
######################################################################
keyword_my_parent = make_keyword_lf(
    keywords=["father", "mother", "mom", "dad", "my mom", "my dad"], label=0
)
```

## Getting Started

Document is quit good and all code in tutorials is easily copied and pasted.






## Conclusion

I'm not sure about huge benefits to using Snorkel in the near future as sole/lead contributor. It did have **some** positive utilization as a sort of diagnostic tool…sort of a heuristic model that told you how good your features/labeling functions are. Of course, it is pretty quick to try out some it might be worth a try.

Its real use seems like it would be in the Mechanical Turk situation. You have a lot of labelings coming from people that are creating a lot of noise (e.g. On the Viability of crowdsourcing NLP annotations in healthcare.)

Thanks for reading!

## Resources

Definitely check out: https://www.snorkel.org/

### YouTube

Newer and older resources videos that contain best practices and some great examples

- Snorkel Workshop 2018: Best Practices for Improving Your Labeling Functions
- Eddie Bell: Weak supervision: a new paradigm for unreliable data | PyData London 2019
- Eddie Bell: Weak supervision: a new paradigm for unreliable data | PyData London 2019

NLP + Data Science ✓ Machine Learning + Deep Learning + Statistics +